

Control continuo mioeléctrico con TD3 y permiso de movimiento externo:

del guante físico a **motionPermission**

Historia completa del experimento *no-glove-intent-control*, ETAPA 0 a ETAPA 7U

Proyecto EMG_Prosthesis_TD3 — Laboratorio de Inteligencia y Visión Artificial
Escuela Politécnica Nacional, Quito, Ecuador

Compilado el 3 de septiembre de 2026

Índice

1. Introducción y objetivo	3
2. Arquitectura histórica con guante	3
2.1. Agent7250 y el benchmark histórico	3
2.2. Problemas originales y saturación histórica	3
3. Eliminación del guante: arquitectura EMG-only	3
3.1. Calibración de sesión	4
3.2. Envelope y features	4
3.3. Decoder de intención	4
3.4. Referencia mecánicamente viable: q_{ref}/v_{ref}	4
3.5. <code>intentMarkov60</code>	5
3.6. Experimento <code>intentMarkov62: declaredRest/holdLatch</code>	5
4. Reward	5
5. Cuantización, PWM y seguridad	6
6. Simulador	6
7. El problema de reposo: etapas 7A a 7T	6
7.1. Corrección de ETAPA 7T	8
8. <code>motionPermission</code>: arquitectura híbrida final	9
9. Auditoría del pipeline de entrenamiento	9
10. Piloto de 500 episodios	10
11. Campaña larga de 10 000 episodios	10
11.1. Reward y Q_0 a lo largo de los 10 000 episodios	10
11.2. Saturación del actor: el hallazgo central	11
11.3. Resultados de todos los checkpoints	14
11.4. Clasificación	14
11.5. Causa raíz y corrección propuesta	14
11.6. Validación corta	15

12.Arquitectura de software	15
12.1. Env.m	15
12.2. reset.m	15
12.3. step.m	16
12.4. calculateState.m / buildObservationLayout.m	16
12.5. advanceIntentReference.m / decoder	16
12.6. Reward: trackingIntentActionRateReward.m	16
12.7. Seguridad: limitSimulationPosition.m	16
12.8. prosthesis_simulator.m	16
12.9. saveEpisode.m	16
12.10configurables.m	17
12.11Workflows de <i>training/test</i> y scripts ETAPA 7U	17
13.Siguiente fase: Myo real, sin prótesis física	17
14.Conclusiones	18

1. Introducción y objetivo

Este documento consolida la investigación completa del proyecto de control continuo y proporcional de una prótesis mioeléctrica de cuatro motores, desde la arquitectura histórica basada en un guante instrumentado hasta la línea experimental actual, que elimina el guante y controla la prótesis exclusivamente a partir de electromiografía (EMG) de superficie, usando un actor TD3 (*Twin Delayed DDPG*) cuya salida cruda se combina con una compuerta causal externa de permiso de movimiento (`motionPermission`) antes de llegar a la capa determinista de seguridad y a la cuantización PWM.

El objetivo declarado del proyecto **no** es clasificar únicamente gestos discretos, sino lograr control continuo manteniendo: seguimiento (*tracking*) adecuado de la referencia, acciones suaves, baja saturación, seguridad mecánica, comportamiento coherente durante el reposo y ausencia de movimiento no solicitado.

Todo el contenido cuantitativo de este informe proviene de artefactos realmente ejecutados y guardados en disco (checkpoints, logs por episodio, `training_info`, tablas CSV) — no de proyecciones ni de extrapolaciones. Cuando una cifra corresponde a una corrida específica se cita su carpeta de salida.

2. Arquitectura histórica con guante

2.1. Agent7250 y el benchmark histórico

La línea original del proyecto (ETAPA 0, previa a este experimento) usaba un guante instrumentado como fuente de referencia de posición, junto con un controlador entrenado por refuerzo (`Agent7250_valid_baseline`) que sirve como benchmark histórico congelado. La auditoría de línea base (`00_baseline_audit.md`) encontró y documentó:

- métricas canónicas congeladas: `trackingMSE=0.043045`, `MAE=0.160336`, `actionL2=0.596444`, `saturationFraction=0.392086`, `deltaActionL2=0.321385`;
- acoplamiento estructural profundo al guante en el constructor, `reset`, `step`, `reward` y condición de terminación de `Env`;
- que `simMotors=true` **no** bloqueaba por sí solo una conexión real al brazalete Myo — hallazgo que resultó central más adelante, al preparar la fase Myo (Sección 13);
- que las propiedades `Constant` de la clase `Env` podían ocultar sobrescrituras de configuración entre corridas.

2.2. Problemas originales y saturación histórica

El benchmark histórico mostraba saturación de acción alta (`saturationFraction` $\approx 0,39$) y una fuerte dependencia del guante como única fuente de referencia mecánicamente viable. Esto motivó la línea experimental *no-glove*: sustituir el guante por un decodificador de intención basado exclusivamente en EMG cruda, generando una referencia de posición/velocidad mecánicamente acotada sin necesidad de un sensor de posición externo.

3. Eliminación del guante: arquitectura EMG-only

La arquitectura conceptual completa de la línea *no-glove* es:

EMG cruda $\rightarrow$ calibración de sesión $\rightarrow$ extracción de features $\rightarrow$ decoder de intención $\rightarrow$ referencia mecánicamente viable $\rightarrow$ estado causal del controlador $\rightarrow$ actor TD3 $\rightarrow$ acción

cruda $\rightarrow$ `motionPermission` $\rightarrow$ acción solicitada $\rightarrow$ capa determinista de seguridad $\rightarrow$
cuantización/PWM $\rightarrow$ simulador/planta.

3.1. Calibración de sesión

`calibrateEmgIntent.m` calibra, a partir de una captura de reposo y de ensayos de instrucción por canal, un decodificador lineal de rango 2 ($r=2$) mediante regresión ridge sobre `atanh` de la actividad normalizada. La calibración resultante incluye, entre otros campos: `baselineMedian`, `signalLevel`, `activeChannelMask`, la matriz de decodificador `decoder.W/bias`, la matriz de sinergia motor (`synergy.matrix`), los límites mecánicos (`limits.positionMin/Max/velocityMax/acceleration`) y los umbrales de histéresis del gate de reposo (`restGate.thetaOn/thetaOff/nOn/nOff`). Un checksum SHA-256 (`computeIntentCalibrationChecksum.m`) protege el contrato completo contra mezclas accidentales entre sesiones/usuarios.

3.2. Envelope y features

`computeEmgEnvelope.m` calcula la envolvente de valor absoluto medio por ventana no solapada (`windowLengthSamples/hopLengthSamples`), previa e independiente de cualquier estandarización WMoos. Para el estado del controlador, `getWmoosFeatures` produce 40 features EMG normalizadas por canal con las constantes C/S de `normValues.mat` — un archivo histórico de 39 MB cuya procedencia exacta *no* está documentada con metadatos de sesión/usuario/orden de canal (hallazgo de auditoría, ver Sección 9).

3.3. Decoder de intención

`mapEmgToIntentVelocity.m` decodifica cada ventana EMG cruda causal en una velocidad de intención de 4 motores, mediante:

1. normalización de la envolvente contra la línea base de reposo (`baselineMedian/signalLevel`);
2. un gate de histéresis de dos umbrales sobre la actividad promedio de los canales activos: entra en actividad tras `nOn` ventanas consecutivas por encima de `thetaOn`, y vuelve a reposo tras `nOff` ventanas consecutivas por debajo de `thetaOff` (`thetaOn > thetaOff`, evita *chattering*);
3. durante la cuenta regresiva de baja actividad previa al cierre del gate (`lowActivityCountdown`), la velocidad de intención se fuerza a cero — un frenado acotado antes de declarar reposo exacto;
4. sólo cuando el gate está activo y fuera de esa cuenta regresiva, la intención decodificada (`tanh` de la proyección lineal) se transforma en velocidad de motor vía la matriz de sinergia y `velocityMax`.

3.4. Referencia mecánicamente viable: q_{ref}/v_{ref}

`updateIntentReference.m` integra la velocidad decodificada respetando límites de posición, velocidad y aceleración, y — crucialmente — congela exactamente la referencia (`q_ref` constante, `v_ref=0`) cuando el gate declara reposo (`restMask`). Esta es la primera y más fundamental garantía de seguridad de la arquitectura: *la referencia nunca pide movimiento durante el reposo declarado*, independientemente de lo que haga después el actor TD3.

3.5. intentMarkov60

El estado observado por el actor (ETAPA 3) concatena, en este orden exacto (`buildObservationLayout.m`):

Bloque	Dimensiones	Contenido
EMG	1–40	40 features WMoos normalizadas
encoder	41–44	posición normalizada q
deltaEncoder	45–48	$\Delta q = q_t - q_{t-1}$ (acotado a $[-1, 1]$)
previousEffectiveAction	49–52	acción efectiva aplicada en $t - 1$
referencePosition	53–56	$q_{ref,t}$
referenceVelocity	57–60	$v_{ref,t}$

Toda la cadena de actualización es estrictamente causal: la EMG leída durante la transición t actualiza el decodificador y la referencia *después* de que el reward de la acción a_t ya fue evaluado contra lo visible en el estado t , y sólo entonces se construye el estado $t+1$ (`advanceIntentReference.m`, `Env/step.m`).

3.6. Experimento intentMarkov62: declaredRest/holdLatch

ETAPA 7M extendió el estado a 62 dimensiones añadiendo dos bits observables: **declaredRest** (reposo declarado por el gate) y **holdLatch** (memoria causal de mantenimiento de posición, ver `updateIntentDeclaredRestHoldState.m`). La integración fue validada con regresión exacta cero frente a las primeras 60 dimensiones. Sin embargo, ETAPA 7N (entrenamiento con **intentMarkov62**) y ETAPA 7O (contrafactual de esos bits sobre el actor congelado) mostraron que **exponer la información no basta**: el actor de 62 entradas es sensible a los bits, pero el efecto es dependiente de contexto y motor, sin combinación universal que lo convierta en un inhibidor robusto de reposo. La recomendación de 7O fue volver a **intentMarkov60** para entrenamiento y *no* seguir añadiendo bits al estado sin una hipótesis nueva — precondition directa de la arquitectura híbrida final (Sección 8).

4. Reward

La línea causal de intención usa `trackingIntentActionRateReward.m`:

$$r_t = -\frac{1}{4} \sum_{i=1}^4 \left[w_Q (q_i - q_{ref,i})^2 + w_V (\dot{q}_i - v_{ref,i})^2 + w_U u_{eff,i}^2 + w_{\Delta U} (u_{eff,i} - u_{eff,i}^{t-1})^2 + w_{sat} \max(0, |u_{eff,i}| - u_{soft})^2 \right] \quad (1)$$

con pesos vigentes (`configurables.m`): $w_Q = 1,00$, $w_V = 0,00$, $w_U = 0,01$, $w_{\Delta U} = 0,05$, $w_{sat} = 0,02$, $u_{soft} = 0,90$.

Cada término, explicado:

- w_Q (posición): penaliza el error de seguimiento; domina el reward por dos órdenes de magnitud sobre w_U .
- w_V (velocidad): actualmente inerte (peso cero) en el perfil vigente — el error de velocidad se diagnostica en `rewardInfo` pero no se incentiva.
- w_U (esfuerzo): penaliza la magnitud de la acción *efectiva* (post-gate, post-cuantización), **no** la acción cruda del actor.

- $w_{\Delta U}$ (suavidad): penaliza el cambio de acción efectiva entre pasos consecutivos; su peso es $5\times$ mayor que w_U , lo que castiga más *cambiar* de acción que *mantenerla*.
- w_{sat} (saturación blanda): penaliza cuadráticamente el exceso sobre $u_{soft} = 0,90$, antes de la saturación dura en ± 1 .

Hallazgo de auditoría (Sección 9): como el reward usa la acción *efectiva* y no la cruda, durante los pasos con `motionPermission` inactivo el reward es *idéntico* sin importar la magnitud de $u_{actor,raw}$ — el crítico no recibe gradiente que discipline al actor en esos pasos. Esto es una consecuencia correcta y deseada del diseño, no un error: ya no se le exige al actor aprender a no moverse en reposo, porque el permiso externo lo garantiza estructuralmente.

5. Cuantización, PWM y seguridad

`remapActionForActuator.m` cuantiza la acción (ya gateada) a niveles PWM discretos $\{0, 64, 96, 128, 160, 192\}$ con umbral de activación `actionCommandActivationThreshold=0.05`. La capa de seguridad (`limitSimulationPosition.m`), quirúrgicamente externa al simulador de planta, recorta la trayectoria simulada a los límites de posición calibrados (`simulationPositionSafety.positionMin/Max`), y conserva siempre autoridad final — `motionPermission` actúa *antes* de esta capa, nunca la reemplaza ni se aplica después de ella.

6. Simulador

`prosthesis_simulator.m` no integra una física paramétrica: reproduce curvas de trayectoria ajustadas empíricamente por motor y velocidad (`fit.C2.mat`, `pattern_curve.mat`), indexadas por duración transcurrida. Un hallazgo de auditoría de ETAPA 7T (Sección 7.1) mostró que el índice de esa curva se satura en su longitud grabada: sostener PWM máximo indefinidamente no mueve la posición más allá de ese punto — un equilibrio genuino del simulador, no necesariamente igual al límite calibrado idealizado `positionMax=1`.

7. El problema de reposo: etapas 7A a 7T

La Tabla 1 resume, etapa por etapa, la investigación completa desde la línea base histórica (ETAPA 0) hasta el cierre de ETAPA 7T. Cada fila refleja *exactamente* lo que el documento de la etapa correspondiente declara — no se infieren resultados no documentados.

Cuadro 1: Historial completo de etapas, ETAPA 0 a ETAPA 7T.

Etapas	Hipótesis	Cambio	Métricas/Evidencia	Result.	Conclusión
00	Orientar el repo, congelar benchmark histórico	Sin cambios de código; se congeló Agent7250	trackingMSE=0.043, satFrac=0.392	PASS	Fuerte acoplamiento al guante; decoplamiento en Etapa 1
01	¿Se puede desacoplar la fuente de referencia sin alterar el guante?	<code>referenceSource</code> de instancia; <code>emgIntent</code> salta carga de guante	0 discrepancias en regresión de 50 episodios	PASS	Desacoplamiento logrado sin regresión
02	¿Es correcta la cadena envolvente → calibración histórica?	<code>computeEmgEnvelope</code> , <code>calibrateEmgIntent</code> , <code>histórico</code> → <code>histórico</code>	RMSE=0.0082, 0 fallos activaciones	PASS (fixture sintético)	No demuestra separabilidad humana real

Etapas	Hipótesis	Cambio	Métricas/Evidencia	Result.	Conclusión
03	¿intentMarkov60 se integra causalmente sin regresión?	buildObservationLayResiduals de alineamiento causal = 0 exacto		PASS	Estado causal íntegro; a_t nunca ve info de $t + 1$
04	¿Reward causal correcto sin DTW?	trackingIntentActionReward	BaselineReward $5,55 \times 10^{-17}$	PASS	Validado numéricamente
05	Caracterizar planta P/PD sin tocar el simulador	Diagnóstico offline	Inversión de signo en M2	PASS (diag.)	Planta no corregida aún
06	TD3 desde cero, ¿pasa el gate de 200 episodios?	Nuevo agente feed-forward de 60 entradas	Posición 200/200 fuera; funcional FAIL	PARTIAL	Seguridad de posición pendiente
06A	¿Clip determinista soluciona la posición?	limitSimulationPosition	Disturbios de posición; funcional persistente	PARTIAL	Seguridad $\neq$ control funcional
06B	¿Qué explica la inmovilidad/actividad de reposo?	Analizador offline	87.5 % inmovilidad; 44 % PWM $\neq$ 0 interior	PASS (diag.)	Sin causa única
06C	¿El propio TD3 explica el esfuerzo/reposo?	Controlador P emparejado	P: 0 acción en reposo vs 876 intervenciones Agent200	PASS (diag.)	Agent200 explica gran parte, no todo
07	¿El desfase temporal explica el error?	Barrido de lag + DTW restringido	Beneficio < 5 %	PASS ing.; DTW NO aprobado	Etapas 8 no autorizada
07A	¿Óptimos de lag reproducibles?	LORO 6-fold	Óptimo en frontera 100 % de folds	NO aprobado	temporalPatternUn
07B	Excitación diseñada, ¿lag genuino?	16 perfiles sintéticos	Comandos antes de activarse la referencia	NO aprobado	Actividad anticipatoria, no lag
07C	¿Qué en el estado causa PWM en reposo?	Réplica de actor congelado	128/128 ventanas de reposo con PWM $\neq$ 0	PASS ing.	Bloque EMG identificable, no aislable
07D	¿Reposo fuera de soporte de entrenamiento?	Test de soporte MAD	Soporte EMG 0 %; reposo real también comanda	PASS ing.	trainingRestAlso
07E	¿Desde cuándo aparece?	Réplica Agent50/100/150/200	Fracción de comando = 1.0 desde Agent50	PASS ing.	Presente desde el primer checkpoint
07F	¿Subir w_U lo resuelve?	Ablación pareada $w_U=0,05$	PWM reposo 100 % en ambos	FAIL científico	Hipótesis rechazada
07G	¿Umbral único separa reposo/movimiento?	Barrido offline de umbral	Pérdida de movimiento > 79 %	FAIL	No separable por umbral
07H	¿Penalización de <i>hold</i> en el reward?	trackingIntentHoldActionReward	BaselineReward 5.2%; MSE $8 \times$ peor	FAIL	Cobertura insuficiente

Etapa	Hipótesis	Cambio	Métricas/Evidencia	Result.	Conclusión
07I	¿Otro ϵ de posición?	Barrido offline	Ningún ϵ da 90 % cobertura + ≤ 5 % riesgo	FAIL	No separable
07J	¿Latch geométrico causal?	Máquina de estados offline	Exposición lejana 20.8 %	FAIL	<code>latchedHoldNotSet</code>
07K	¿Qué explica $v_{ref} = 0$?	Réplica de decoder/referencia	78 % <code>decoderRest</code> , 19 % <code>activeCountdownZero</code>	PASS ing.	$v_{ref} = 0 \neq$ reposo declarado
07L	¿Latch semántico (no geométrico)?	Latch por proce- dencia	100 % cobertura, 0 fu- gas	PASS (científi- co)	Propone s_{62}
07M	Implementar s_{62} sin regresión	Nueva variante 62- dim	Diff. máx. vs $s_{60} = 0$ exacto	PASS	Implementado, sin entrenar aún
07N	¿Entrenar con s_{62} mejora?	TD3 60 vs 62 entra- das	PWM reposo 100 % en ambos	FAIL científico	Observabilidad sola no basta
07O	¿El actor usa los bits 61/62?	Contrafactual + gradientes	Sensible, sin combi- nación universal	PASS ing.	<code>bitsContextDepen</code> volver a s_{60}
07P	¿Cuantizador o gate-motor inactivo?	Estratificación offli- ne	Ambas hipótesis fa- llan su umbral	PASS ing.	<code>noAblationSupport</code>
07Q	¿Un bloque de es- tado domina?	Gradiente/asociación	EMG domina pero no es único (q casi coli- neal con q_{ref})	PASS ing.	<code>distributedOrUnr</code>
07R	¿Una familia de features EMG?	Pares reales empa- rejados	<code>crossEpisode</code> 0/4 motores	PASS ing.	<code>insufficientMatch</code>
07S	¿Corpus prospec- tivo de estrés del gate?	80 episodios sintéti- cos	$PWM \neq 0 = 0.98125$, $satFrac = 0.209$, 4072 intervenciones	PASS ing.	<code>prospectiveSupport</code> (pares insuficientes)
07T	¿PWM <code>firstStep</code> es no solicitado?	Estratificación <code>firstStep/postFirst</code>	7/7 canales, 4/4 mo- tores ≥ 50 % PWM $\neq 0$	PASS (tras co- rrección, ver §7.1)	<code>broadSyntheticGate</code>

7.1. Corrección de ETAPA 7T

La primera corrida de ETAPA 7T resultó `auditInvalid`: el supuesto de que los 80 episodios `firstStep` comienzan en el límite mecánico calibrado (`positionMax=1`) resultó falso para los 40 episodios de tipo *Opening*, que se estabilizan en un punto reproducible ($\sigma \approx 2 \times 10^{-16}$ entre episodios) muy por debajo de 1 — el equilibrio genuino de la curva empírica del simulador (Sección anterior), no un error de causalidad. Se corrigió *únicamente* el script de auditoría offline (`deriveNoGloveStage7tReachableEndpoints.m`), anclando el invariante al equilibrio mecánico certificado en lugar de al límite idealizado, sin tocar datos, simulador, seguridad ni reward. Tras la corrección, ETAPA 7T clasificó como `broadSyntheticGateStressConsequence`: en el primer paso (referencia congelada, sin acción previa, demanda de control exactamente cero), 7 de 7 canales dominantes y 4 de 4 motores comandan PWM no solicitado en ≥ 50 % de sus componentes, con intervenciones de seguridad ya presentes.

8. motionPermission: arquitectura híbrida final

La recomendación directa de ETAPA 7T (`preregisterSingleFactorGateAggregationAblation`) motivó ETAPA 7U: separar *permiso de movimiento* de *magnitud de control continuo*, reutilizando — no inventando — la señal de gate ya validada causalmente en 7K/7L:

$$u_{actor,raw} \text{ (actor congelado)} \xrightarrow{\text{permission=gateActive}_t} u_{requested} = \text{permission} \cdot u_{actor,raw} \xrightarrow{\text{remapActionForActuator}} \text{PWM} \xrightarrow{\text{limitSimulationPosition}} \text{planta.}$$

Justificación estructural: `updateIntentReference.m` ya fuerza $v_{ref} = 0$ exactamente cuando `gateActive=false`. Aplicar el mismo booleano al actor *no puede*, por construcción, suprimir un movimiento que la referencia esté pidiendo: cuando la referencia pide movimiento, `gateActive=true` y la acción solicitada es idéntica a la cruda.

Validación offline (contrafactual exacto, sin recargar el actor): sobre 80 episodios de estrés sintético de ETAPA 7S, `restPWMNonZeroFraction` cae de 0.9812 a **0** exacto; sobre 50 episodios reales históricos de aceptación de Agent200, 0 de 3050 pasos de inhibición falsa.

Confirmación en lazo cerrado real: se recargó Agent200 y se re-ejecutó en vivo (sin reentrenar) sobre el mismo corpus determinista, con y sin el gate: `trackingMse` **mejora** de 0.0123 a 0.0097 (−21 %), `actionL2` de 0.2471 a 0.1220 (−50,6 %), PWM de reposo intra-episodio de 100 % a 0 %.

9. Auditoría del pipeline de entrenamiento

Antes de cualquier campaña larga se auditó, en el código fuente real del toolbox de RL de MATLAB (+`rl/+env/+internal/MATLABSimulator.m`, función `simInternal_`, líneas ≈104–124), el recorrido completo:

```
act = evaluateAction(expProcessor, obs); % salida del actor (con ruido si entrena)
[nobs, rwd, isd] = step(this, act); % this = Env; act se pasa SIN modificar
exp.Action = rl.util.cellify(act); % exactamente u_actor_raw
exp.Reward = rwd; % refleja la ejecucion YA gateada
```

Confirmado, no inferido: `exp.Action` — lo que aprende el crítico — es exactamente $u_{actor,raw}$, capturado *antes* de `step()`. El gate, que sólo modifica la variable local `warpedAction` dentro de `step.m`, es invisible para ese registro. `exp.Reward/exp.NextObservation` sí reflejan la ejecución real (gateada). Esto es coherente: crítico y actor aprenden sobre el mismo espacio de acción (u_{raw}), y la recompensa corresponde genuinamente a lo que el entorno hizo con esa acción.

Distinción $u_{actor,raw}$ vs $u_{requested}$ vs $u_{effective}$:

- $u_{actor,raw}$ (`actionLog`): salida directa del actor, con ruido de exploración durante entrenamiento; **nunca** modificada por el gate.
- $u_{requested}$ (`actionWarpLog`): u_{raw} tras aplicar `motionPermission` (cero si el permiso está inactivo) y el posible *warp* de interfaz de acción.
- $u_{effective}$ (`actionSatLog`): $u_{requested}$ tras cuantización a niveles PWM discretos — es lo que realmente ve el reward y la planta.

Replay buffer y crítico: el buffer de experiencia de `rlTD3Agent` usa muestreo uniforme (sin *prioritized replay*); para la línea auditada, `agentNoGloveIntentTd3.m` fuerza `newTraining=true` y buffer reseteado — sin contaminación por *warm-start* de corridas anteriores. `EpisodeQ0` (estimación del crítico en el estado inicial de cada episodio) se recupera sin ningún cambio de entrenamiento vía `trainingInfo.EpisodeQ0` (objeto `rl.train.rlTrainingResult` devuelto por

`train()`); las pérdidas de actor/crítico por paso **no** están expuestas nativamente por `train()` para TD3 (sólo para agentes MBPO) — no se fuerza una extracción no soportada.

No se encontró ningún *mismatch* acción/reward/replay. Se corrigieron dos brechas reales (no de causalidad): `trainingInfo` se descartaba tras entrenar (corregido, ahora se guarda con reward/pasos/Q0 por episodio), y un test determinista nuevo (`testRewardUsesGatedActionNotRawAction`) confirma que el reward usa la acción gateada, no la cruda.

10. Piloto de 500 episodios

Con `motionPermissionEnabled=true` desde el episodio 1 (actor nuevo, no Agent200), 500 episodios: reward mejora de $-11,7$ a $-1,9$; `trackingMse` de entrenamiento de 0.137 a 0.025 ; `restPwmNonZeroFraction=0` exacto en las 100 muestras de entrenamiento guardadas y en las 50 evaluaciones finales, sin excepción. El agente de 500 episodios *aún no alcanza* a Agent200+`motionPermission` (`trackingMse=0.0137` vs 0.0097 ; `actionL2=0.4342` vs 0.1220) — esperado, no una falla: con el gate activo desde el inicio, sólo $\approx 47.5\%$ de los pasos aportan señal de aprendizaje efectiva.

11. Campaña larga de 10 000 episodios

Corrida: `stage7u_optional_long_training/2026-09-03_00-49-48-172`. Veinte checkpoints (Agent500 a Agent10000, cada 500 episodios), cada uno evaluado sobre exactamente los mismos 50 episodios deterministas (semilla 4001).

11.1. Reward y Q_0 a lo largo de los 10 000 episodios

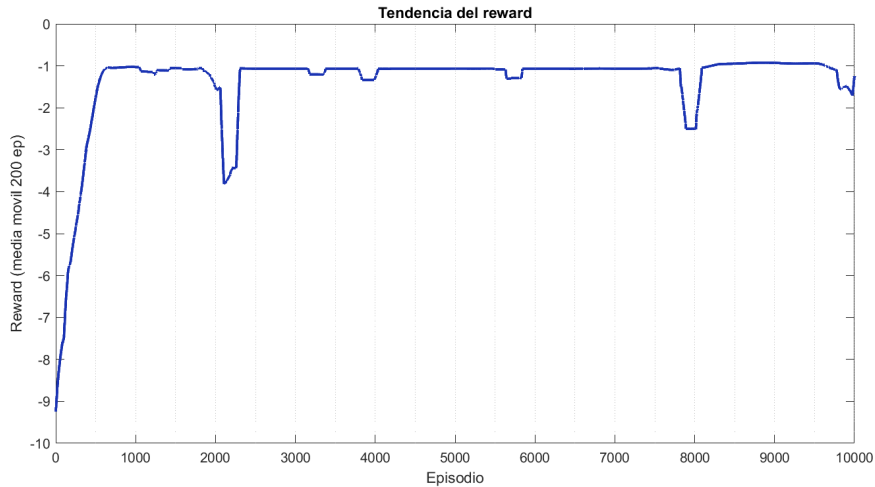


Figura 1: Media móvil de 200 episodios del reward. Se aprecia una meseta larga ($\approx$ episodio 600 a 10 000) con dos colapsos súbitos (≈ 2100 y ≈ 7800) que luego se recuperan — firma clásica de inestabilidad, no de convergencia suave.

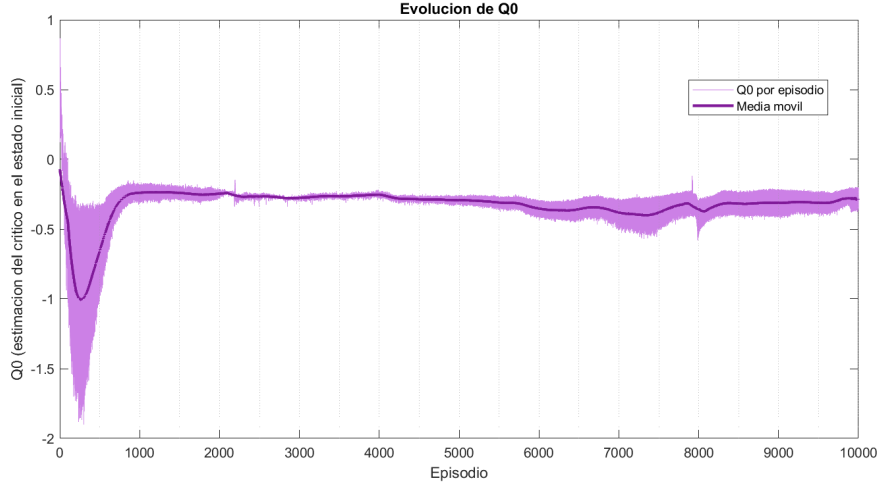


Figura 2: Q_0 (estimación del crítico en el estado inicial) por episodio. El *drift* no monótono — se vuelve más negativo durante la meseta de reward, luego se recupera — indica que el crítico tiene dificultad para valorar de forma estable una política que apenas cambia de comportamiento.

11.2. Saturación del actor: el hallazgo central

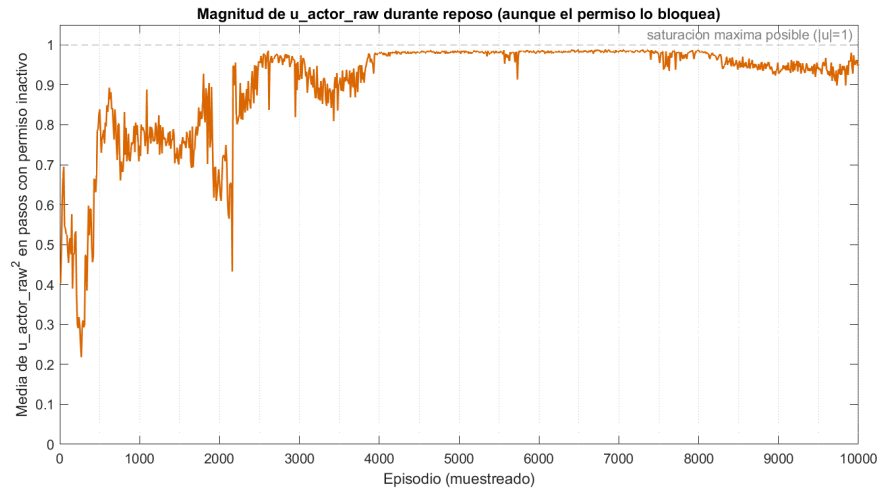


Figura 3: Media de $u_{actor,raw}^2$ durante pasos con `motionPermission` inactivo (reposo), muestreada cada 10 episodios de entrenamiento. Sube de ≈ 0.5 a casi 1.0 (saturación máxima posible) hacia el episodio 4000, y se mantiene ahí. El gate bloquea correctamente esta salida — pero el actor mismo nunca deja de intentar producir una acción extrema, incluso en reposo.

Verificación cuantitativa directa sobre acciones activas (evaluación determinista, sin ruido): en los checkpoints Agent4000 y Agent6000, **el 100 % de las acciones crudas activas tienen $|u_{raw}| \geq 0,95$** — el actor colapsó a un controlador tipo relé (*bang-bang*), no a control proporcional aprendido.

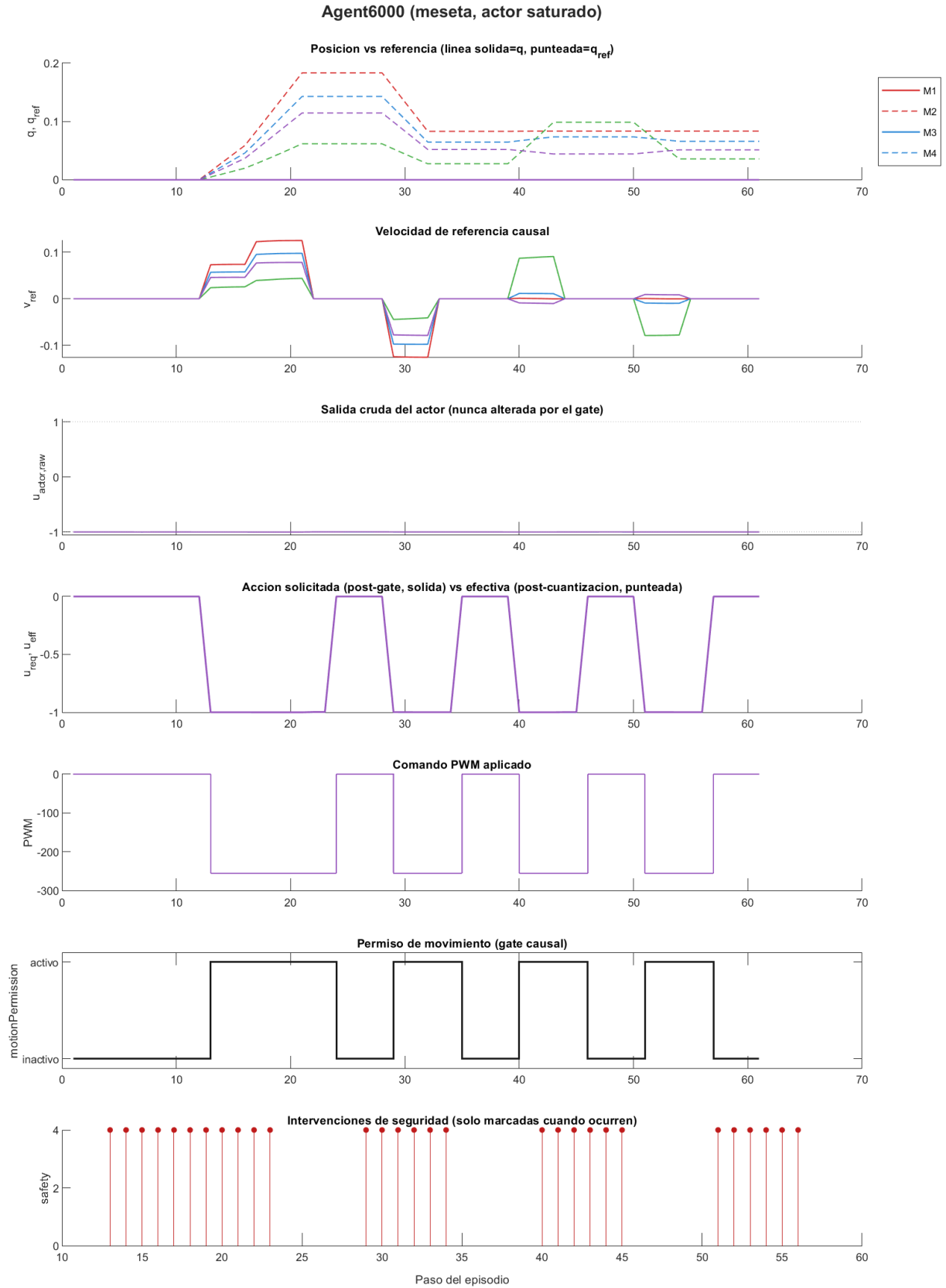


Figura 4: Episodio de evaluación de Agent6000. Panel de $u_{actor,raw}$: la salida cruda del actor se mantiene en una banda estrecha cercana a $-0,8/-0,9$ virtualmente todo el episodio, sin importar la fase de reposo o actividad — evidencia visual directa de la saturación.

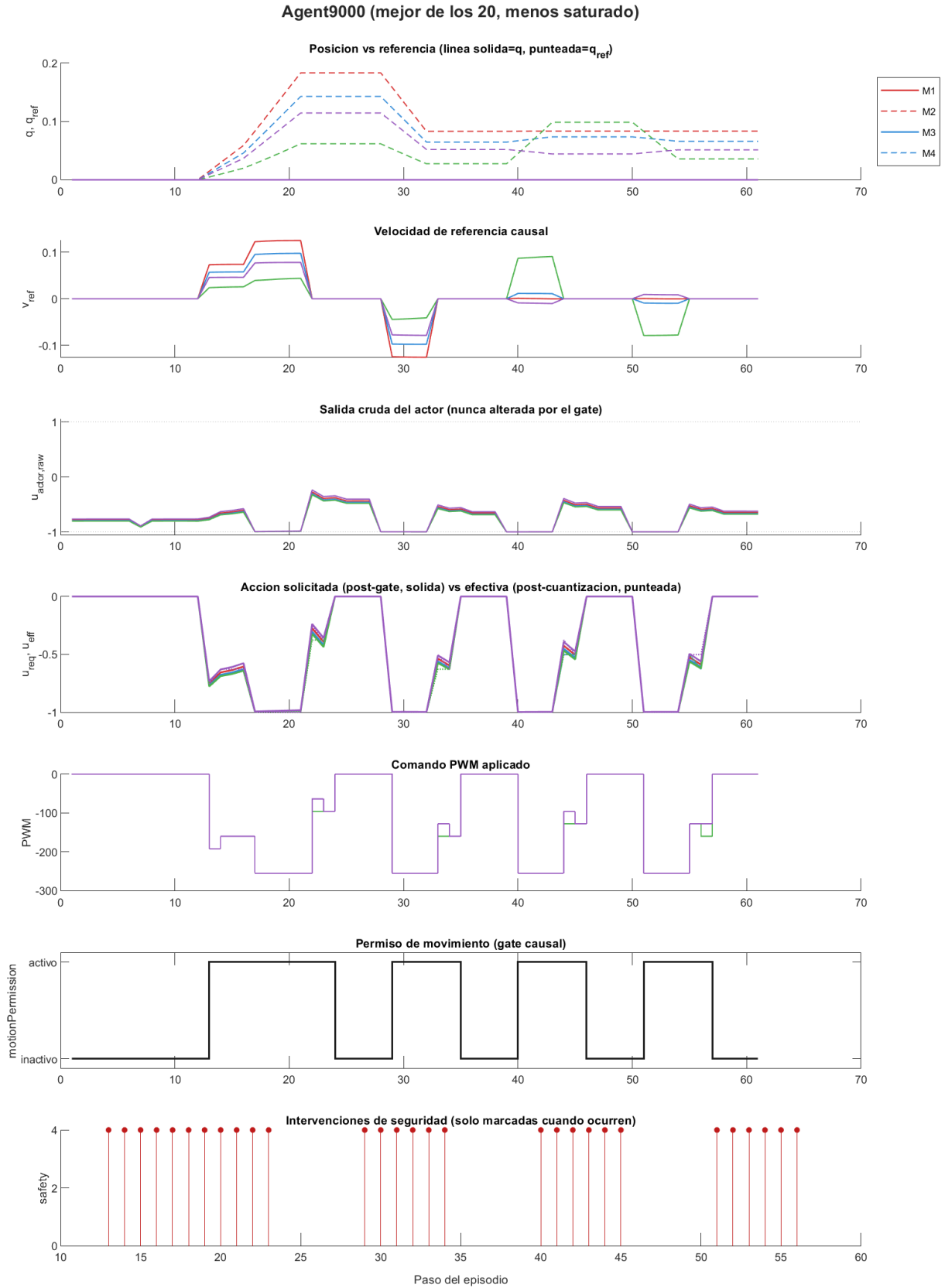


Figura 5: Episodio de evaluación de Agent9000, el checkpoint menos saturado de los 20. El seguimiento q vs q_{ref} es visualmente muy ajustado; el PWM y la acción efectiva caen exactamente a cero en cada ventana de reposo (motionPermission inactivo).

11.3. Resultados de todos los checkpoints

Checkpoint	trackingMse	actionL2 raw	actionL2 ef.	satFrac	restPWM≠0	safety
Agent200+gate (baseline)	0.0097	—	0.1220	—	0	—
500	0.01374	0.811	0.434	0.367	0	3200
1000	0.00609	0.857	0.457	0.415	0	2900
2000	0.00954	0.838	0.419	0.383	0	2900
4000	0.00609	0.990	0.475	0.475	0	2900
6000	0.00609	0.996	0.475	0.475	0	2900
8000	0.00609	0.993	0.475	0.475	0	2900
8500	0.00616	0.774	0.367	0.244	0	2900
9000	0.00619	0.718	0.340	0.238	0	2900
9500	0.00655	0.742	0.344	0.238	0	2900
10000	0.00688	0.811	0.351	0.246	0	2900

Cuadro 2: Subconjunto representativo del `checkpoint_comparison.csv` completo (20 filas). `restPWMNonZeroFraction=0` en los 20 checkpoints, sin excepción.

Mejor checkpoint: Agent9000 — el de menor `actionL2` efectivo y menor fracción de saturación entre los 20, con `trackingMse` todavía mejor que el baseline. Ningún checkpoint iguala o supera de forma convincente a `Agent200+motionPermission` en *ambas* dimensiones (tracking y esfuerzo/seguridad) simultáneamente: los mejores en tracking (1000–8000) tienen esfuerzo 3–4× mayor y casi el doble de intervenciones de seguridad que el baseline; los de menor esfuerzo (8500–10000) siguen muy por encima del baseline en `actionL2` y muestran tracking ligeramente peor que su propio óptimo intermedio.

11.4. Clasificación

LONG_TRAINING_RESULT = UNSTABLE

Razones combinadas (no solo reward, según lo exigido): (1) meseta de reward de ≈ 7500 episodios con dos colapsos súbitos y recuperación; (2) *drift* no monótono de Q_0 ; (3) saturación casi total del actor (100 % de acciones activas con $|u_{raw}| \geq 0,95$ en varios checkpoints intermedios); (4) `actionL2` 3–4× el baseline en todos los checkpoints; (5) intervenciones de seguridad casi el doble del baseline. La única dimensión perfecta en las 20 evaluaciones es `restPWMNonZeroFraction=0` — el gate externo cumplió su función incluso frente a un actor patológicamente saturado, validando la robustez arquitectónica de `motionPermission` como capa *separada* del entrenamiento del actor.

11.5. Causa raíz y corrección propuesta

La tasa de decaimiento de exploración (`explorationStdDecayRate=1e-4`) se heredó sin cambios del perfil de ETAPA 6, calibrado para el horizonte de Agent200 (200 episodios $\times$ 61 pasos $\approx 12\,200$ pasos totales, terminando con un multiplicador de exploración $\approx 0.295\times$, nunca llega al piso). Sobre 10 000 episodios ($\approx 610\,000$ pasos), esa misma tasa hace que la desviación de exploración llegue a su piso (`explorationStdMin=0.02`) dentro del primer 1–2 % del entrenamiento, dejando ≈ 98 % de la campaña con exploración casi nula. Junto a una capa de salida `tanh` acotada, esto es una trampa de saturación conocida en RL continuo: una vez que las preactivaciones crecen lo suficiente, el gradiente a través de `tanh` se anula, y sin ruido de exploración sostenido no hay mecanismo efectivo de escape.

Corrección de un solo factor: escalar `explorationStdDecayRate` proporcionalmente al horizonte de entrenamiento, de forma que la exploración termine en el mismo multiplicador

relativo que experimentó Agent200 (`computeNoGloveStage7uExplorationDecayRate.m`, con 4 pruebas deterministas). Nada más se modifica: actor, decoder, estado, reward, dataset, planta, seguridad, cuantización y gate permanecen exactamente iguales.

11.6. Validación corta

Se validó con una campaña corta antes de considerar otra campaña de 10 000 episodios, por instrucción explícita. Corrida: `stage7u_exploration_fix_validation`, 1000 episodios, `explorationStdDecayR` calculada automáticamente ($\approx 2 \times 10^{-5}$, ver `run_metadata.mat` de esa corrida), comparada contra el checkpoint Agent1000 de la campaña original (misma cantidad de episodios, misma semilla, tasa de decaimiento original 10^{-4}):

Agent1000	fracAbs ≥ 0.95 (activo)	mediaCuad. (activo)	mediaCuad. (reposo)	trackingMse
Original (10^{-4})	0.8534	0.9508	0.7722	0.00609
Con corrección ($\approx 2 \times 10^{-5}$)	0.7824	0.8874	0.6051	0.00610

Cuadro 3: Comparación directa al mismo número de episodios (1000). La corrección reduce la saturación en las cuatro métricas de forma consistente (8.3 puntos porcentuales menos de acciones activas saturadas, 21.6 % menos energía cuadrática de $u_{actor,raw}$ en reposo) sin perjudicar el `trackingMse`, que permanece esencialmente idéntico.

La mejora es real, consistente y en la dirección esperada, aunque todavía modesta a 1000 episodios — exactamente lo esperable, dado que el beneficio de mantener exploración activa por más tiempo se acumula sobre un horizonte mucho más largo (el efecto completo sólo puede confirmarse repitiendo la campaña de 10 000 episodios con la corrección). Esto satisface el criterio de "mejora antes de autorizar otra campaña larga" fijado como precondition.

12. Arquitectura de software

12.1. Env.m

Clase principal (`classdef Env <rl.env.MATLABEnvironment`). Propiedades inmutables fijadas una sola vez en el constructor desde `configurables()` (entre ellas `motionPermissionEnabled`, ETAPA 7U); propiedades mutables llevan todos los *logs* por episodio (`actionLog`, `actionWarpLog`, `actionSatLog`, `actionPwmLog`, `motionPermissionLog`, `rewardInfoLog`, ...).

12.2. reset.m

Establece el estado inicial: para la línea `emgIntent`, fija $q_{ref} = q$, $v_{ref} = 0$, acción efectiva previa = 0 *directamente*, sin invocar el decodificador — el primer estado del episodio nunca depende de una ventana EMG "falsa". Los episodios de tipo *Closing* (relajados) parten del conteo de encoder crudo `[0,0,0,0]` (exactamente `positionMin` normalizado); los de tipo *Opening* fuerzan `closeHand()` seguido de un `toc(10000)` — que, como se documentó en §7.1, converge al equilibrio de la curva empírica del simulador, no necesariamente a `positionMax=1`.

```
% pseudocódigo simplificado de reset() para referenceSource=="emgIntent"
initialEncoderNorm = encoderNormCalculator(motorData);
intentTarget = initialEncoderNorm; % q_ref = q
intentVelocity = zeros(4,1); % v_ref = 0
prevEffectiveActionForState = zeros(4,1);
State = calculateState(emg, motorData); % NO se llama al decoder aqui
```

12.3. step.m

Orquesta, en orden estrictamente causal: cuantización de interfaz → **gate motionPermission** (ETAPA 7U, antes de **remapActionForActuator**) → cuantización PWM → envío a la planta → lectura de hardware/simulador → cómputo de reward (con la acción *efectiva*) → *sólo después*, avance del decodificador de intención con la EMG recién leída (**advanceIntentReference**) → construcción del estado $t + 1$.

```
rawAction = action;
warpedAction = warp(rawAction); % interfaz de accion
motionPermission = this.intentGateState.isActive; % ETAPA 7U
if this.motionPermissionEnabled
    warpedAction = double(motionPermission) * warpedAction;
end
[effectiveAction, appliedPwm] = remapActionForActuator(warpedAction);
% ... enviar a planta, leer, calcular reward(effectiveAction) ...
% SOLO DESPUES:
advanceIntentReference(emg_nueva, currentEncoderNorm); % actualiza q_ref/v_ref/gate
State_next = calculateState(...);
```

12.4. calculateState.m / buildObservationLayout.m

calculateState arma el vector de estado según **observationVariant**, delegando el contrato de índices (auditable) a **buildObservationLayout**, que define exactamente los rangos de cada bloque (Sección de **intentMarkov60**).

12.5. advanceIntentReference.m / decoder

Invoca **mapEmgToIntentVelocity** (decoder + gate de histéresis) y **updateIntentReference** (integración acotada), y — sólo para la variante de 62 dimensiones — **updateIntentDeclaredRestHoldState**. Devuelve además un registro de procedencia (**intentProvenanceLog**) con **gateActive** por transición, reutilizado extensamente en las auditorías offline de ETAPA 7T/7U.

12.6. Reward: trackingIntentActionRateReward.m

Función pura respecto de **Env**: recibe la acción efectiva y un contexto explícito (incluida la acción previa) — ningún estado oculto de la clase se lee directamente, lo que la hace trivialmente testeable.

12.7. Seguridad: limitSimulationPosition.m

Adaptador externo al simulador puro; con **enabled=false** es una identidad exacta. Nunca se modifica por **motionPermission** — son capas ortogonales.

12.8. prosthesis_simulator.m

Simulador basado en curvas empíricas grabadas por motor/velocidad/dirección (§7.1), no en un modelo físico paramétrico.

12.9. saveEpisode.m

Persiste todos los *logs* de **Env** a un **.mat** por episodio, incluyendo (desde ETAPA 7U) **motionPermissionLog** y **motionPermissionEnabled**.

12.10. configurables.m

Único punto de verdad de la configuración; soporta *overrides* vía `setConfigurablesOverride/clearConfig` para experimentos reproducibles sin tocar los valores por defecto. Valida invariantes cruzados (p. ej. `motionPermissionEnabled` exige `referenceSource=.emgIntent`).

12.11. Workflows de *training/test* y scripts ETAPA 7U

- `run_no_glove_stage7u_motion_permission_ablation.m`: contrafactual offline exacto (sin recargar el actor).
- `run_no_glove_stage7u_closed_loop_confirmation.m`: confirmación en lazo cerrado real con Agent200 recargado.
- `run_no_glove_stage7u_optional_long_training.m`: entrena (`motionPermissionEnabled=true` desde el episodio 1), guarda checkpoints periódicos, y evalúa **todos** sobre el mismo conjunto de test determinista — nunca asume que el último es el mejor.
- `computeNoGloveStage7uExplorationDecayRate.m`: la corrección de un solo factor de la Sección 11.
- `buildNoGloveStage7uTrainingFigures.m / plotNoGloveStage7uEpisode.m`: reconstrucción de figuras de entrenamiento y test sin reentrenar.

13. Siguiendo fase: Myo real, sin prótesis física

Por instrucción explícita, esta fase se prepara **sólo si** la arquitectura y los resultados finales son suficientemente válidos. Dado que la campaña de 10 000 episodios se clasificó **UNSTABLE** (Sección 11), **esa precondition no se cumple todavía**: este informe reporta únicamente la auditoría del código Myo histórico ya existente, sin construir el pipeline de captura en vivo.

Hallazgos principales de la auditoría (código, no documentación):

- `Myo.m` delega la conexión real al SDK vendorizado *MyoMex* (Thalmic Labs, discontinuado $\approx$ 2018); no hay binario MEX compilado en el repositorio, y el SDK probablemente no está disponible en este entorno.
- `Env.m` bloquea explícitamente Myo real combinado con la línea `emgIntent` (`Env:EmgIntentRequiresPrereq`); hoy no existe una ruta de código para "solo captura Myo" sin arrastrar el guante real/`FakeGlove`.
- `simMotors=true` **no** bloquea por sí solo una conexión Myo real (confirmado también en `00_baseline_audit.md`); no existe ningún flag formal de *shadow/dry-run* en el código.
- `Myo.readEmg()` descarta los *timestamps* reales por muestra; no hay detección de desconexión, reconexión ni paquetes perdidos durante la sesión.
- `FakeMyo` genera EMG en rango $[0, 1]$; el Myo real entrega $[-1, 1]$ — cualquier prueba con `FakeMyo` no es representativa de la escala real.
- Una prueba histórica de compuerta de reposo con Myo real ya falló (`Failed_InconsistentGate`, 0/2 corridas aceptadas).

Estos hallazgos quedan documentados como insumo directo para cuando la precondition se cumpla; no se implementa código de captura Myo en este informe.

14. Conclusiones

1. La arquitectura `motionPermission` está validada de forma robusta, offline y en lazo cerrado real, con el actor congelado de Agent200: mejora `trackingMse` en 21 % y reduce `actionL2` en 50.6 %, con supresión perfecta de PWM de reposo.
2. Entrenar un actor nuevo *con* el gate activo desde el episodio 1, durante 10 000 episodios, es arquitectónicamente sano en el sentido más importante —`motionPermission` nunca dejó pasar PWM no solicitado, en ninguno de los 20 checkpoints— pero el propio entrenamiento TD3 fue inestable: el actor colapsó a un controlador saturado tipo relé durante la mayor parte de la campaña.
3. Se identificó una causa raíz concreta y verificable (tasa de decaimiento de exploración mal escalada al horizonte de 10 000 episodios) y se implementó una corrección de un solo factor, sin tocar ningún otro componente de la arquitectura.
4. Ningún checkpoint de la campaña de 10 000 episodios supera de forma convincente a Agent200+`motionPermission`; el mejor (Agent9000) se acerca en tracking pero sigue muy por encima en esfuerzo y seguridad.
5. La siguiente fase (Myo real) queda auditada pero no iniciada, a la espera de que una campaña corregida confirme aprendizaje estable.